Predicting breast cancer from digitized images of breast mass

Tiffany A. Timbers, Joel Ostblom & Melissa Lee

2023-11-09

```
import numpy as np
import pandas as pd
import requests
import zipfile
import altair as alt
from sklearn import set_config
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import make_column_transformer, make_column_selector
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import fbeta_score, make_scorer

# png, so the charts also show in the PDF
_ = alt.renderers.enable("png", scale_factor=2)
```

Summary

Here we attempt to build a classification model using the k-nearest neighbours algorithm which can use breast cancer tumour image measurements to predict whether a newly discovered breast cancer tumour is benign (i.e., is not harmful and does not require treatment) or malignant (i.e., is harmful and requires treatment intervention). Our final classifier performed fairly well on an unseen test data set, with the F2 score, where beta = 2, of 0.96 and an overall accuracy calculated to be 0.96. On the 171 test data cases, it correctly predicted 168. It incorrectly predicted 3 cases, which were all false positives - predicting that a tumour is malignant when in fact it is benign. These kind of incorrect predictions is not as harmful as a false negative in our context. Although they could theoretically cause the patient to undergo unnecessary treatment if the model is used as a decision tool, it is likely that the model is used for initial screening and that there will be a follow up appointment and further testing until treatment commences. As such, we believe this model is at, or close to, the performance required for it to have clinical utility, although further research

to improve the model performance and understand the characteristics of incorrectly predicted patients would still be useful.

Introduction

Women have a 12.1% lifetime probability of developing breast cancer, and although cancer treatment has improved over the last 30 years, the projected death rate for women's breast cancer is 22.4 deaths per 100,000 in 2019 (Canadian Cancer Statistics Advisory Committee 2019). Early detection has been shown to improve outcomes (Canadian Cancer Statistics Advisory Committee 2019), and thus methods, assays and technologies that help to improve diagnosis may be beneficial for improving outcomes further.

Here we ask if we can use a machine learning algorithm to predict whether a newly discovered tumour is benign or malignant given tumour image measurements. Answering this question is important because traditional methods for tumour diagnosis are quite subjective and can depend on the diagnosing physicians skill as well as experience (Street, Wolberg, and Mangasarian 1993). Furthermore, benign tumours are not normally dangerous; the cells stay in the same place and the tumour stops growing before it gets very large. By contrast, in malignant tumours, the cells invade the surrounding tissue and spread into nearby organs where they can cause serious damage. Thus, if a machine learning algorithm can accurately and effectively predict whether a newly discovered tumour benign or malignant given tumour image measurements this could lead to less subjective, and more scalable breast cancer tumour diagnosis which could contribute to better patient outcomes.

Methods

Data

The data set used in this project is of digitized breast cancer image features created by Dr. William H. Wolberg, W. Nick Street, and Olvi L. Mangasarian at the University of Wisconsin, Madison (Street, Wolberg, and Mangasarian 1993). It was sourced from the UCI Machine Learning Repository (Street, Wolberg, and Mangasarian 1993) and can be found here, specifically this file. Each row in the data set represents summary statistics from measurements of an image of a tumour sample, including the diagnosis (benign or malignant) and several other measurements (e.g., nucleus texture, perimeter, area, etc.). Diagnosis for each image was conducted by physicians.

Analysis

The k-nearest neighbors (k-nn) algorithm was used to build a classification model to predict whether a tumour mass was benign or malignant (found in the class column of the data set). All variables included in the original data set, with the exception of the standard error of fractal dimension, smoothness, symmetry and texture were used to fit the model. Data was split with 70% being partitioned into the training set and 30% being partitioned into the test set. The hyperparameter K was chosen using 30-fold cross validation with the F2 score as the classification metric. Beta was chosen to be set to 2 for the F2 score to increase the weight on recall during fitting because the application is cancer screening and false negatives are very undesirable in such an

application. All variables were standardized just prior to model fitting. The Python programming language (Van Rossum and Drake 2009) and the following Python packages were used to perform the analysis: requests(Reitz 2011), zipfile (Van Rossum and Drake 2009), numpy(Harris et al. 2020), Pandas (McKinney 2010), altair (VanderPlas, 2018), scikit-learn (Pedregosa et al. 2011). The code used to perform the analysis and create this report can be found here: https://github.com/UBC-MDS/DSCI_522_project-template-breast-cancer-predictor.

Results & Discussion

To look at whether each of the predictors might be useful to predict the tumour class, we plotted the distributions of each predictor from the training data set and coloured the distribution by class (benign: blue and malignant: orange). In doing this we see that class distributions for all of the mean and max predictors for all the measurements overlap somewhat, but do show quite a difference in their centres and spreads. This is less so for the standard error (se) predictors. In particular, the standard errors of fractal dimension, smoothness, symmetry and texture look very similar in both the distribution centre and spread. Thus, we choose to omit these from our model.

```
# download data as zip and extract
url  =  "https://archive.ics.uci.edu/static/public/15/breast+cancer+wisconsin+
original.zip"

request = requests.get(url)
with open("../data/raw/breast+cancer+wisconsin+original.zip", 'wb') as f:
    f.write(request.content)

with  zipfile.ZipFile("../data/raw/breast+cancer+wisconsin+original.zip",  'r')
as zip_ref:
    zip_ref.extractall("../data/raw")
```

```
# pre-process data (e.g., scale and split into train & test)
# read in data
colnames = [
    "id",
    "class",
    "mean_radius",
    "mean_texture",
    "mean_perimeter",
    "mean_area",
    "mean_smoothness",
    "mean_compactness",
    "mean_concavity",
    "mean_concave_points",
    "mean_symmetry",
```

```
    "mean_fractal_dimension",
    "se_radius",
    "se_texture",
    "se_perimeter",
    "se_area",
    "se_smoothness",
    "se_compactness",
    "se_concavity",
    "se_concave_points",
    "se_symmetry",
    "se_fractal_dimension",
    "max_radius",
    "max_texture",
    "max_perimeter",
    "max_area",
    "max_smoothness",
    "max_compactness",
    "max_concavity",
    "max_concave_points",
    "max_symmetry",
    "max_fractal_dimension"
]

cancer       =       pd.read_csv("../data/raw/wdbc.data",       names=colnames,
header=None).drop(columns=['id'])
# re-label Class 'M' as 'Malignant', and Class 'B' as 'Benign'
cancer['class'] = cancer['class'].replace({
    'M' : 'Malignant',
    'B' : 'Benign'
})
cancer
```

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	mean_compactness	mean_concavity	mean_concave_points	mean_symmetry	mean_fractal_dimension	se_radius	se_texture	se_perimeter	se_area	se_smoothness	se_compactness	se_concavity	se_concave_points	se_symmetry	se_fractal_dimension	max_radius	max_texture	max_perimeter	max_area	max_smoothness	max_compactness	max_concavity	max_concave_points	max_symmetry	max_fractal_dimension
0	Malignant	17.99	10.38	122.8	1001	0.1184	0.2776	0.3001	0.1471	0.2419	0.07871	1.095	0.9053	8.589	153.4	0.006399	0.04904	0.05373	0.01587	0.03003	0.006193	25.38	17.33	184.6	2019	0.1622	0.6656	0.7119	0.2654	0.4601	0.1189
1	Malignant	20.57	17.77	132.9	1326	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	0.5435	0.7339	3.398	74.08	0.005225	0.01308	0.0186	0.0134	0.01389	0.003532	24.99	23.41	158.8	1956	0.1238	0.1866	0.2416	0.186	0.275	0.08902

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	mean_compactness	mean_concavity	mean_concave_points	mean_symmetry	mean_fractal_dimension	radius_error	texture_error	perimeter_error	area_error	smoothness_error	compactness_error	concavity_error	concave_points_error	symmetry_error	fractal_dimension_error	worst_radius	worst_texture	worst_perimeter	worst_area	worst_smoothness	worst_compactness	worst_concavity	worst_concave_points	worst_symmetry	worst_fractal_dimension	class
2	19.69	21.25	130.0	1203.0	0.1096	0.1599	0.1974	0.1279	0.2069	0.0599	0.7456	0.7869	4.585	94.03	0.00615	0.04006	0.03832	0.02058	0.0225	0.004571	23.57	25.53	152.5	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	Malignant
3	11.42	20.38	77.58	386.1	0.1425	0.2839	0.2414	0.1052	0.2597	0.09744	0.4956	1.156	3.445	27.23	0.00911	0.07458	0.05661	0.01867	0.05963	0.009208	14.91	26.5	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	Malignant
4	20.29	14.34	135.1	1297.0	0.1003	0.1328	0.198	0.1043	0.1809	0.05883	0.7572	0.7813	5.438	94.44	0.01149	0.02461	0.05688	0.01885	0.01756	0.005115	22.54	16.67	152.2	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678	Malignant
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
564	21.56	22.39	142.0	1479.0	0.1111	0.1159	0.2439	0.1389	0.1726	0.05623	1.176	1.256	7.673	158.7	0.0103	0.02891	0.05198	0.02454	0.01114	0.004239	25.45	26.4	166.1	2027.0	0.1411	0.2113	0.4107	0.2216	0.2060	0.07115	Malignant
565	20.13	28.25	131.2	1261.0	0.0978	0.1034	0.1441	0.09791	0.1752	0.05533	0.7655	2.463	5.203	99.04	0.005769	0.02423	0.0395	0.01678	0.01898	0.002498	23.69	38.25	155.0	1731.0	0.1166	0.1922	0.3215	0.1628	0.2572	0.06637	Malignant
566	16.60	28.08	108.3	858.1	0.0846	0.1023	0.0992	0.0533	0.1590	0.05648	0.4564	1.075	3.425	48.55	0.005903	0.03731	0.0473	0.01557	0.01318	0.003892	18.98	34.12	126.7	1124.0	0.1139	0.3094	0.3403	0.1418	0.2218	0.07820	Malignant
567	20.60	29.33	140.1	1265.0	0.1178	0.2770	0.3514	0.1520	0.2397	0.07016	0.7260	1.305	5.270	99.14	0.007510	0.06411	0.07545	0.03116	0.02556	0.005080	25.74	39.42	184.6	1821.0	0.1650	0.8681	0.9387	0.2650	0.4087	0.12400	Malignant
568	7.76	24.54	47.92	181.0	0.0526	0.0436	0.02	0.0	0.1587	0.06529	0.2371	2.04	1.674	9.981	0.008415	0.01307	0.01232	0.0	0.02005	0.004144	9.456	30.37	59.16	268.6	0.0892	0.0964	0.0771	0.0	0.2742	0.07039	Benign

```
np.random.seed(522)
set_config(transform_output="pandas")

# create the split
cancer_train, cancer_test = train_test_split(
    cancer, train_size=0.70, stratify=cancer["class"]
)

cancer_train.to_csv("../data/processed/cancer_train.csv")
cancer_test.to_csv("../data/processed/cancer_test.csv")
```

```
cancer_preprocessor = make_column_transformer(
    (StandardScaler(), make_column_selector(dtype_include='number')),
    remainder='passthrough',
    verbose_feature_names_out=False
)

cancer_preprocessor.fit(cancer_train)
scaled_cancer_train = cancer_preprocessor.transform(cancer_train)
scaled_cancer_test = cancer_preprocessor.transform(cancer_test)

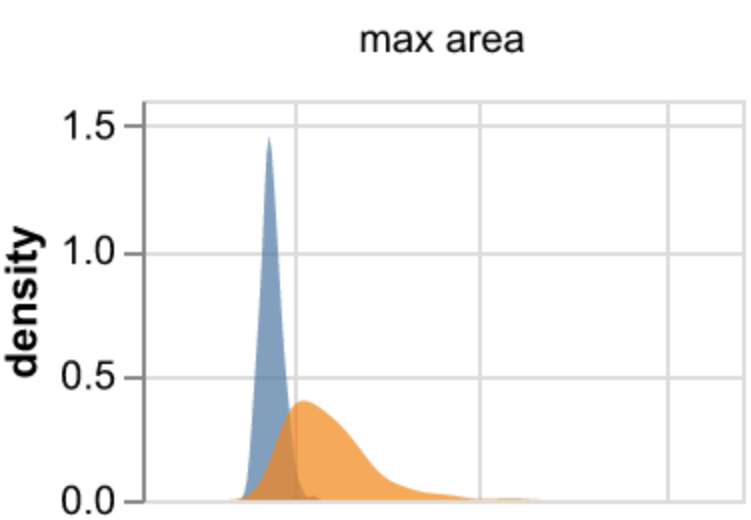
scaled_cancer_train.to_csv("../data/processed/scaled_cancer_train.csv")
scaled_cancer_test.to_csv("../data/processed/scaled_cancer_test.csv")
```

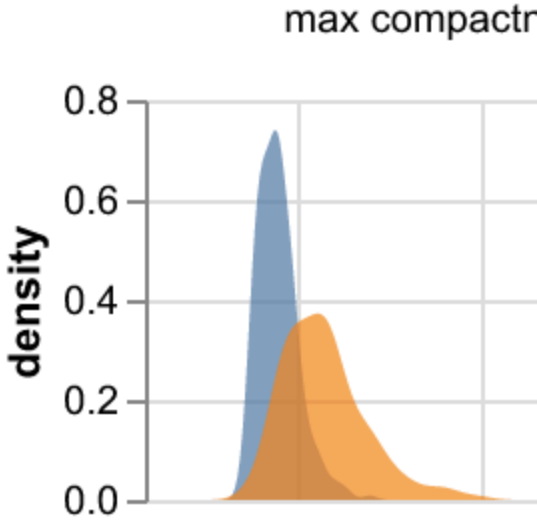
```
# melt for plotting via facets
cancer_train_melted = scaled_cancer_train.melt(
    id_vars=['class'],
    var_name='predictor',
    value_name='value'
)

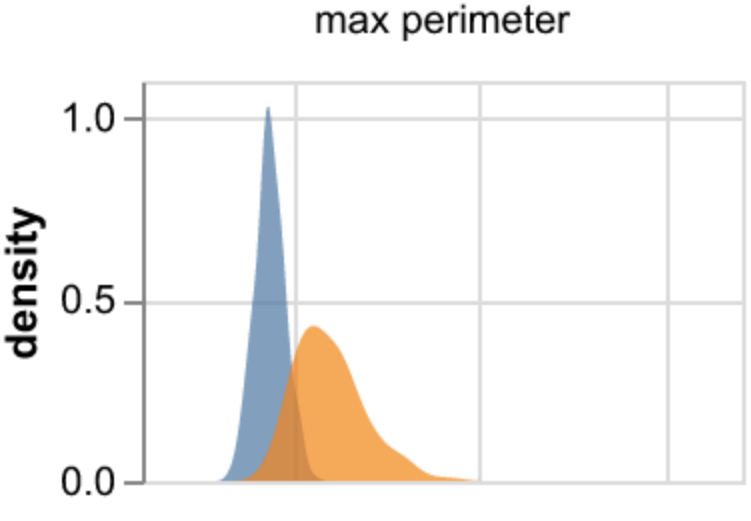
# make columns names nicer for plotting
cancer_train_melted['predictor']                                        =
cancer_train_melted['predictor'].str.replace('_',' ')
```

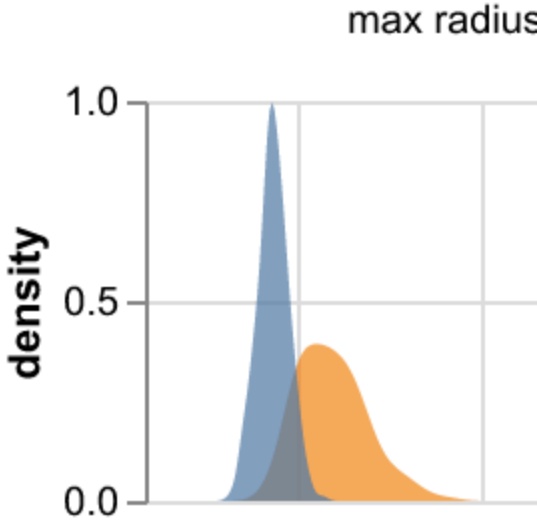
```
# exploratory data analysis - visualize predictor distributions across classes
alt.data_transformers.enable('vegafusion')

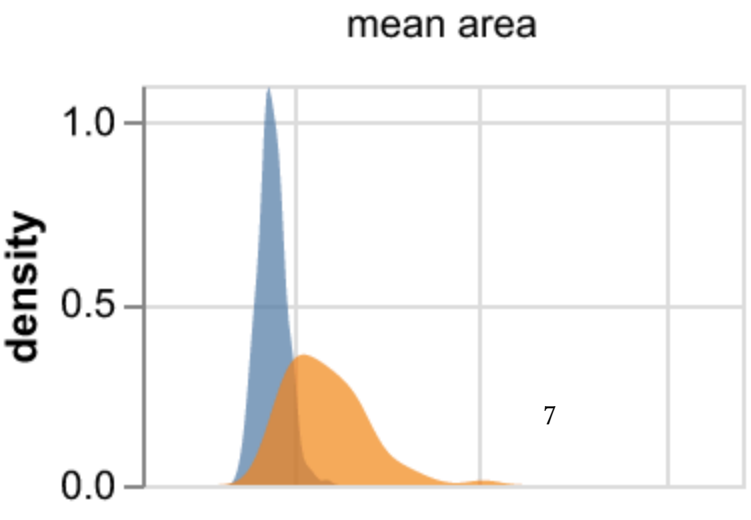
alt.Chart(cancer_train_melted, width=150, height=100).transform_density(
    'value',
    groupby=['class', 'predictor']
).mark_area(opacity=0.7).encode(
    x="value:Q",
    y=alt.Y('density:Q').stack(False),
    color='class:N'
).facet(
    'predictor:N',
    columns=5
).resolve_scale(
    y='independent'
)
```


max area
density
1.5
1.0
0.5
0.0


max compactn
density
0.8
0.6
0.4
0.2
0.0


max perimeter
density
1.0
0.5
0.0


max radius
density
1.0
0.5
0.0


mean area
density
1.0
0.5
0.0

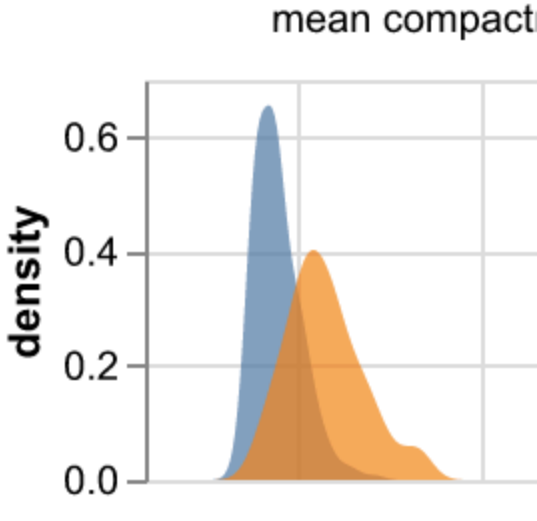

mean compact
density
0.6
0.4
0.2
0.0

Figure 1. Comparison of the empirical distributions of training data predictors between benign and malignant tumour masses.

```
# drop se_smoothness, se_symmetry, se_texture
cancer_train   =   cancer_train.drop(columns=["se_smoothness",   "se_symmetry",
"se_texture", "se_fractal_dimension"])
```

We chose to use a simple classification model using the k-nearest neighbours algorithm. To find the model that best predicted whether a tumour was benign or malignant, we performed 30-fold cross validation using F2 score (beta = 2) as our metric of model prediction performance to select K (number of nearest neighbours). We observed that the optimal K was 4.

```
# tune model (here, find K for k-nn using 30 fold cv)
knn = KNeighborsClassifier()
cancer_tune_pipe = make_pipeline(cancer_preprocessor, knn)

parameter_grid = {
    "kneighborsclassifier__n_neighbors": range(1, 100, 3),
}

cv = 30
cancer_tune_grid = GridSearchCV(
    estimator=cancer_tune_pipe,
    param_grid=parameter_grid,
    cv=cv,
    scoring=make_scorer(fbeta_score, pos_label='Malignant', beta=2)
)
```

```
cancer_fit = cancer_tune_grid.fit(
    cancer_train.drop(columns=["class"]),
    cancer_train["class"]
)

accuracies_grid = pd.DataFrame(cancer_fit.cv_results_)
```

```
accuracies_grid = (
    accuracies_grid[[
        "param_kneighborsclassifier__n_neighbors",
        "mean_test_score",
        "std_test_score"
    ]]
    .assign(
```

```
        sem_test_score=accuracies_grid["std_test_score"] / cv**(1/2),
       # `lambda` allows access to the chained dataframe so that we can use the
newly created `sem_test_score` column
                   sem_test_score_lower=lambda  df:  df["mean_test_score"]  -
(df["sem_test_score"]/2),
                   sem_test_score_upper=lambda  df:  df["mean_test_score"]  +
(df["sem_test_score"]/2)
    )
    .rename(columns={"param_kneighborsclassifier__n_neighbors": "n_neighbors"})
    .drop(columns=["std_test_score"])
)

accuracies_grid.sort_values("mean_test_score", ascending=False).head(10)
```

	n_neighbors	mean_test_score	sem_test_score	sem_test_score_lower	sem_test_score_upper
1	4	0.922786	0.019224	0.913174	0.932398
2	7	0.921674	0.019426	0.911961	0.931388
0	1	0.911018	0.022412	0.899812	0.922224
4	13	0.904803	0.019468	0.895069	0.914537
7	22	0.895741	0.021242	0.885120	0.906362
9	28	0.894630	0.021379	0.883940	0.905319
10	31	0.894630	0.021379	0.883940	0.905319
3	10	0.893897	0.018998	0.884397	0.903396
8	25	0.893348	0.021205	0.882745	0.903950
11	34	0.889702	0.022619	0.878393	0.901012

```
line_n_point                                                                =
alt.Chart(accuracies_grid, width=600).mark_line(color="black").encode(
    x=alt.X("n_neighbors").title("Neighbors"),
    y=alt.Y("mean_test_score")
        .scale(zero=False)
        .title("F2 score (beta = 2)")
)

error_bar = alt.Chart(accuracies_grid).mark_errorbar().encode(
     alt.Y("sem_test_score_upper:Q").scale(zero=False).title("F2 score (beta =
2)"),
    alt.Y2("sem_test_score_lower:Q"),
```

```
    alt.X("n_neighbors:Q").title("Neighbors")
)

line_n_point + line_n_point.mark_circle(color='black') + error_bar
```

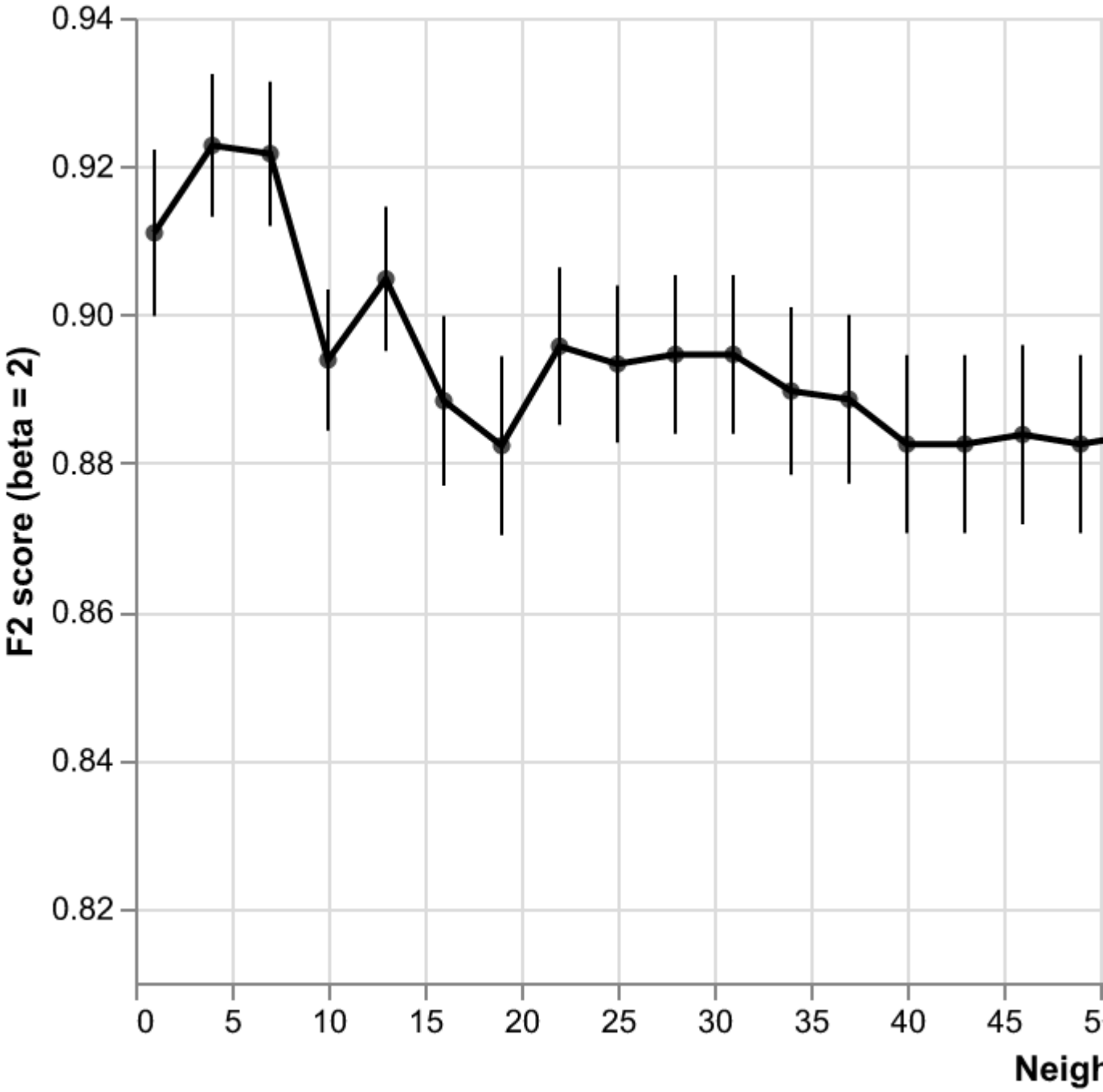


Figure 2. Results from 30-fold cross validation to choose K. F2 score (with beta = 2) was used as the classification metric as K was varied.

```
# Compute accuracy
accuracy = cancer_fit.score(
    cancer_test.drop(columns=["class"]),
    cancer_test["class"]
)

# Compute F2 score (beta = 2)
cancer_preds = cancer_test.assign(
    predicted=cancer_fit.predict(cancer_test)
)
f2_beta_2_score = fbeta_score(
    cancer_preds['class'],
    cancer_preds['predicted'],
    beta=2,
    pos_label='Malignant'
)

pd.DataFrame({'accuracy': [accuracy], 'F2 score (beta = 2)': [f2_beta_2_score]})
```

	accuracy	F2 score (beta = 2)
0	0.962145	0.962145

Our prediction model performed quite well on test data, with a final overall accuracy of 0.96 and F2 (beta = 2) score of 0.98. Other indicators that our model performed well come from the confusion matrix, where it only made 3 mistakes. All 3 mistakes were predicting a benign tumour as malignant, which is promising for implementing this in the clinic as false positives are less harmful than false negatives.

Table 1. Confusion matrix of model performance on test data.

```
pd.crosstab(
    cancer_preds["class"],
    cancer_preds["predicted"],
)
```

predicted class	Benign	Malignant
Benign	107	0
Malignant	3	61

While the performance of this model is likely already useful as a screening tool in a clinical setting, there are several directions that could be explored for to improve it further. First, we could look closely at the 3 misclassified observations and compare them to several observations that were classified correctly (from both classes). The goal of this would be to see which feature(s) may be driving the misclassification and explore whether any feature engineering could be used to help the model better predict on observations that it currently is making mistakes on. Additionally, we would try seeing whether we can get improved predictions using other classifiers. One classifier we might try is random forest forest because it automatically allows for feature interaction, where k-nn does not. Finally, we also might improve the usability of the model in the clinic if we output and report the probability estimates for predictions. If we cannot prevent misclassifications through the approaches suggested above, at least reporting a probability estimates for predictions would allow the clinician to know how confident the model was in its prediction. Thus the clinician may then have the ability to perform additional diagnostic assays if the probability estimates for prediction of a given tumour class is not very high.

References

Canadian Cancer Statistics Advisory Committee. 2019. "Canadian Cancer Statistics." Canadian Cancer Society. http://cancer.ca/Canadian-Cancer-Statistics-2019-EN.

Dua, Dheeru, and Casey Graff. 2017. "UCI Machine Learning Repository." University of California, Irvine, School of Information; Computer Sciences. http://archive.ics.uci.edu/ml.

Harris, C.R. et al., 2020. Array programming with NumPy. Nature, 585, pp.357–362.

Jed Wing, Max Kuhn. Contributions from, Steve Weston, Andre Williams, Chris Keefer, Allan Engelhardt, Tony Cooper, Zachary Mayer, et al. 2019. Caret: Classification and Regression Training. https://CRAN.R-project.org/package=caret.

McKinney, Wes. 2010. "Data Structures for Statistical Computing in Python." In Proceedings of the 9th Python in Science Conference, edited by Stéfan van der Walt and Jarrod Millman, 51–56.

Pedregosa, F. et al., 2011. Scikit-learn: Machine learning in Python. Journal of machine learning research, 12(Oct), pp.2825–2830.

Reitz, Kenneth. 2011. Requests: HTTP for Humans. https://requests.readthedocs.io/en/master/.

Street, W. Nick, W. H. Wolberg, and O. L. Mangasarian. 1993. "Nuclear feature extraction for breast tumor diagnosis." In Biomedical Image Processing and Biomedical Visualization, edited by Raj S. Acharya and Dmitry B. Goldgof, 1905:861–70. International Society for Optics; Photonics; SPIE. https://doi.org/10.1117/12.148698.

VanderPlas, J. et al., 2018. Altair: Interactive statistical visualizations for python. Journal of open source software, 3(32), p.1057.

Van Rossum, Guido, and Fred L. Drake. 2009. Python 3 Reference Manual. Scotts Valley, CA: CreateSpace.